1. ENUNCIADO GENERAL

En el primer examen parcial, desarrollaron un prototipo funcional de una plataforma inteligente de comercio electrónico denominada FashionStore, destinada a una cadena de tiendas de ropa con múltiples sucursales.

La plataforma contempla administración de productos, categorías, tallas, colores, temporadas, proveedores, inventarios, clientes, reservas, ventas presenciales y digitales, pagos, realidad aumentada, inteligencia artificial y aplicaciones web y móvil.

Para el segundo examen parcial, la empresa responsable de FashionStore ha decidido evolucionar su solución tecnológica hacia un nuevo modelo de negocio y operación.

La empresa ya no pretende utilizar FashionStore exclusivamente para administrar sus propias tiendas. Su nuevo objetivo es comercializar FashionStore como una plataforma de software bajo el modelo SaaS (Software as a Service), permitiendo que diferentes empresas dedicadas a la comercialización de prendas de vestir contraten el servicio mediante planes de suscripción.

Cada empresa suscrita deberá disponer de su propio entorno lógico de trabajo, administrar sus sucursales, empleados, clientes, productos, inventarios, reservas y ventas, sin acceder a la información de otras empresas.

Adicionalmente, debido a que muchas tiendas pueden experimentar problemas de conectividad a Internet, FashionStore deberá permitir que determinadas operaciones comerciales continúen funcionando sin conexión, tanto desde la aplicación web como desde la aplicación móvil.

La plataforma deberá incorporar mecanismos de almacenamiento local, registro de operaciones pendientes, sincronización posterior y tratamiento de posibles conflictos de información.

La empresa también desea potenciar sus capacidades de inteligencia artificial mediante la incorporación de reportes generativos, que permitan a los usuarios autorizados solicitar información empresarial utilizando lenguaje natural, obteniendo reportes construidos a partir de los datos reales de su empresa.

Finalmente, debido a los cambios funcionales y arquitectónicos, el equipo de desarrollo deberá aplicar la metodología ICONIX, actualizando y ampliando los modelos UML pertinentes, y garantizando coherencia entre requisitos, análisis, diseño, implementación y pruebas.

El propósito del segundo parcial no consiste en desarrollar una nueva aplicación desde cero, sino en evolucionar, adaptar e integrar nuevas capacidades al prototipo construido durante el primer examen parcial.

2. OBJETIVO GENERAL

Evolucionar la plataforma FashionStore hacia un plataforma funcional de comercio electrónico SaaS multitenant, incorporando planes de suscripción, aplicaciones web progresivas, operación offline en web y móvil, sincronización de datos y reportes generativos mediante inteligencia artificial, aplicando la metodología ICONIX y garantizando la coherencia entre los modelos UML 2.5+ y la implementación.

3. OBJETIVOS ESPECÍFICOS

1. Aplicar ICONIX al análisis, diseño y evolución del sistema.
2. Transformar FashionStore en una plataforma multitenant que permita administrar múltiples empresas independientes.
3. Implementar planes de suscripción que regulen el acceso a funcionalidades y recursos.
4. Evolucionar la aplicación Angular hacia una Progressive Web App (PWA).
5. Incorporar persistencia local y operaciones offline en la aplicación web.
6. Incorporar SQLite como mecanismo de persistencia local en Flutter.
7. Permitir el registro de reservas y ventas sin conectividad a Internet.
8. Implementar sincronización de operaciones pendientes entre las aplicaciones cliente y el backend.
9. Detectar y gestionar conflictos derivados de operaciones offline.
10. Incorporar reportes generativos mediante inteligencia artificial y lenguaje natural.
11. Garantizar aislamiento y seguridad de información entre empresas.
12. Implementar pruebas funcionales, de integración y de reglas críticas de negocio.
13. Actualizar la documentación técnica y UML2.5+ para reflejar fielmente el software desarrollado.

4. DESAFÍO I — APLICACIÓN DE LA METODOLOGÍA ICONIX

4.1. Descripción

Durante el primer parcial se utilizó el Proceso Unificado de Desarrollo de Software (PUDS). En el segundo parcial, los estudiantes deberán aplicar ICONIX, un proceso de desarrollo dirigido por casos de uso, que establece una relación explícita entre requisitos, análisis, diseño e implementación. La aplicación de ICONIX deberá concentrarse principalmente en los nuevos requisitos y en los componentes existentes que deban modificarse.

5. DESAFÍO II — APLICACIÓN WEB PROGRESIVA (PWA)

5.1. Descripción

La aplicación web desarrollada con Angular deberá evolucionar hacia una Progressive Web App, incorporando capacidades de instalación, almacenamiento local y funcionamiento parcial sin conexión a Internet. La aplicación deberá poder continuar ejecutando determinadas operaciones comerciales cuando el dispositivo pierda conectividad.

5.2. Características obligatorias

- Ser instalable en navegadores compatibles.
- Conservar los recursos necesarios para ejecutar las funciones offline.
- Permitir consultar información previamente almacenada.
- Registrar operaciones comerciales localmente.
- Identificar operaciones pendientes de sincronización.
- Permitir sincronización al recuperar conectividad.
- Informar al usuario el estado de sus operaciones.

5.3. Operaciones offline obligatorias

1. Consultar productos previamente descargados.
2. Consultar precios, tallas, colores y disponibilidad local conocida.
3. Registrar una reserva de prendas.
4. Registrar una venta de contado.
5. Consultar las reservas y ventas locales pendientes.
6. Conservar la información después de cerrar y volver a abrir la aplicación.

La aplicación deberá indicar cuando la información consultada pueda estar desactualizada.

5.4. Sincronización

Cuando exista conectividad, el sistema deberá permitir enviar las operaciones pendientes al backend. Las operaciones deberán disponer de identificadores únicos y estados que permitan reconocer si fueron registradas localmente, enviadas, confirmadas o rechazadas. No se aceptará que una operación desaparezca del almacenamiento local simplemente porque se intentó enviarla al servidor.

6. DESAFÍO III — APLICACIÓN MÓVIL OFFLINE CON FLUTTER Y SQLITE

6.1. Descripción

La aplicación móvil Flutter deberá incorporar persistencia local utilizando SQLite, permitiendo continuar determinadas operaciones de FashionStore sin depender permanentemente del backend. La aplicación deberá funcionar bajo un esquema de operación local y sincronización posterior.

6.2. Funcionalidades obligatorias

- Descargar y almacenar un subconjunto del catálogo.
- Consultar productos, precios y existencias conocidas.
- Registrar reservas sin conexión.
- Registrar ventas de contado sin conexión.
- Consultar operaciones pendientes.
- Reiniciar la aplicación sin perder los datos registrados.
- Sincronizar operaciones cuando se restablezca Internet.
- Consultar el resultado de la sincronización.

6.3. Persistencia local

- Productos y variantes necesarias para operar.
- Información de sucursales autorizadas.
- Datos mínimos de clientes requeridos para las operaciones.
- Ventas y sus detalles.
- Reservas y sus detalles.
- Operaciones pendientes de sincronización.
- Estados y resultados de sincronización.

El diseño deberá evitar duplicación innecesaria y conservar la integridad referencial apropiada.

6.4. Reglas de sincronización

- Identificación única de operaciones.
- Asociación de cada operación a su empresa y sucursal.
- Reintentos ante fallos de comunicación.
- Prevención de duplicaciones.
- Confirmación de operaciones aceptadas.
- Identificación de operaciones rechazadas o en conflicto.

La PWA y Flutter podrán compartir contratos API y reglas de negocio, aunque utilicen mecanismos de almacenamiento local diferentes.

7. DESAFÍO IV — VENTAS OFFLINE Y CONSISTENCIA DE INVENTARIO

7.1. Descripción

La venta offline constituye un proceso crítico porque la información local de inventario puede no coincidir con la información central. El sistema deberá permitir registrar ventas locales y posteriormente intentar confirmarlas en el servidor, respetando las reglas comerciales definidas.

7.2. Registro de venta

- Identificador único.
- Empresa y sucursal.

- Usuario responsable.
- Cliente, cuando corresponda.
- Fecha y hora de registro.
- Productos y variantes.
- Cantidades.
- Precios utilizados.
- Total de la operación.
- Forma de pago registrada.
- Estado de sincronización.
- **Gestionar cambios de prendas de vestir en un determinado tiempo**
- **Otras funcionalidades como aportes propio**

Para el alcance obligatorio, las ventas offline serán de contado y no requerirán procesamiento electrónico del pago mientras no exista conexión. La confirmación local de recepción de efectivo no implica necesariamente la aceptación definitiva de la venta por el servidor.

8. DESAFÍO V — REPORTES GENERATIVOS MEDIANTE INTELIGENCIA ARTIFICIAL

8.1. Descripción

FashionStore deberá incorporar una funcionalidad que permita a usuarios autorizados solicitar reportes empresariales utilizando lenguaje natural. El sistema deberá interpretar la solicitud, identificar los datos necesarios, ejecutar consultas controladas y presentar los resultados. No será suficiente integrar un chatbot de preguntas generales porque los reportes pueden tener varios criterios de selección y necesidad de consultar a mas de una tabla, y mostrarlo en pantalla, exportar a Excel o pdf con un formato de reporte convencional pero que se construye dinámicamente según requerimiento del cliente .

8.2. Ejemplos de solicitudes

- Mostrame las ventas de septiembre agrupadas por sucursal.
- ¿Cuáles fueron los diez productos más vendidos este mes?
- Generá un gráfico de ventas por semana.
- Compará las ventas de las sucursales durante los últimos tres meses.
- Mostrame los productos con menor existencia.
- ¿Qué categorías generaron mayores ingresos durante este período?
- Reportes tipos da

8.3. Requisitos obligatorios

- Recibir solicitudes mediante lenguaje natural escrito o verbal (voz).
- Interpretar el tipo de reporte solicitado.
- Identificar filtros como fechas, sucursales, categorías y productos.
- Obtener información real de la base de datos.

- Generar una representación adecuada del reporte.
- Permitir presentar resultados en tablas o gráficos.
- Restringir la información al tenant autorizado.
- Manejar solicitudes ambiguas, incompletas o no soportadas.
- Evitar consultas arbitrarias no autorizadas.

Se exigirá demostrar al menos cuatro tipos diferentes de reportes, con filtros variables y datos reales. No se aceptarán resultados estáticos preparados exclusivamente para la demostración.

8.4 **Desarrollo del módulo de analítica inteligente mediante Machine Learning**

- Generación y preparación de datos sintéticos
- Selección de variables y preprocesamiento de datos
- Desarrollo y entrenamiento de modelos supervisados
- Desarrollo y entrenamiento de modelos no supervisados
- Evaluación y validación de los modelos entrenados
- Integración de los modelos con el sistema
- Generación de reportes inteligentes
- Desarrollo del dashboard analítico
- Visualización e interpretación de resultados

8.5. Integración de componentes

Alcance del módulo de analítica inteligente

- **Datos sintéticos:** generación de un conjunto de datos artificiales para entrenamiento y evaluación.
- **Aprendizaje supervisado:** entrenamiento de un modelo para predecir una variable objetivo conocida.
- **Aprendizaje no supervisado:** entrenamiento de un modelo para identificar grupos, patrones o anomalías sin etiquetas previas.
- **Reportes inteligentes:** presentación de predicciones, patrones detectados y métricas de evaluación.
- **Dashboard analítico:** visualización de indicadores, gráficos y resultados de los modelos.

8.4. Seguridad

La IA no deberá tener acceso irrestricto a la base de datos. El backend deberá validar las operaciones permitidas, los parámetros de consulta y el alcance de información correspondiente al usuario autenticado. El modelo de IA podrá ser local o consumido mediante API, conforme a las posibilidades técnicas del grupo.

8.5. Extensiones

- Solicitudes mediante voz.
- Respuestas narrativas generadas por IA.
- Gráficos seleccionados dinámicamente.
- Exportación de reportes.
- Comparaciones y análisis más elaborados.

Estas extensiones no reemplazan los requisitos obligatorios.

9. DESAFÍO VI — TRANSFORMACIÓN A SAAS MULTITENANT

9.1. Descripción

FashionStore deberá evolucionar desde una solución para una sola cadena de tiendas hacia una plataforma SaaS utilizada por múltiples empresas independientes. Cada empresa contratante será considerada un tenant. Un tenant podrá disponer de múltiples sucursales, empleados, clientes, productos, inventarios, reservas y ventas.

9.2. Modelo de datos

Para el examen se utilizará una base de datos PostgreSQL compartida, con aislamiento lógico de información por tenant. Los estudiantes deberán identificar las entidades que pertenecen a cada tenant y establecer los mecanismos necesarios para impedir accesos cruzados.

- Plataforma SaaS.
- Empresa contratante o tenant.
- Sucursal de una empresa.
- Usuarios y roles.
- Plan contratado.
- Suscripción.
- Recursos y operaciones comerciales.

Una sucursal no representa un tenant. Varias sucursales pueden pertenecer a una misma empresa.

9.3. Administración de empresas

- Registrar empresas contratantes.
- Identificar un administrador por empresa.
- Asociar usuarios y sucursales.
- Consultar el plan contratado.
- Administrar el estado de la suscripción.
- Restringir funcionalidades según el plan.

Deberán existir, como mínimo, dos empresas de prueba independientes.

9.4. Planes de suscripción

Característica	Básico	Profesional
Sucursales activas	Hasta 2	Hasta 10
Gestión de productos	Sí	Sí
Gestión de ventas	Sí	Sí
Reservas	Sí	Sí
Operación offline	Sí	Sí
Reportes convencionales	Sí	Sí
Reportes generativos con IA	No	Sí

Los límites son valores establecidos para el prototipo académico y deberán ser configurables desde el backend.

9.5. Gestión de suscripciones

7. Asignar un plan a una empresa.
8. Registrar fecha de inicio y vencimiento.
9. Consultar el estado de la suscripción.
10. Cambiar de plan.
11. Verificar límites y funcionalidades habilitadas.
12. Restringir operaciones según la suscripción.

Para este parcial no será obligatorio implementar cobro recurrente real ni integración con plataformas externas de facturación SaaS. Se permitirá simular administrativamente la contratación y renovación de planes.

9.6. Seguridad multitenant

Un usuario de la Empresa A no podrá consultar, modificar o eliminar información perteneciente a la Empresa B. Esta restricción deberá aplicarse en el backend, no solamente ocultando opciones en las interfaces. El sistema deberá comprobar la autorización del usuario y la pertenencia de los recursos al tenant correspondiente.

10. REQUISITOS NO FUNCIONALES OBLIGATORIOS

RNF01. Seguridad

Los mecanismos de autenticación y autorización deberán proteger el acceso a recursos y operaciones.

RNF02. Aislamiento de datos

Ninguna operación deberá permitir acceder a información de otro tenant sin autorización.

RNF03. Persistencia offline

Las operaciones registradas sin conexión deberán sobrevivir al cierre y reinicio normal de la aplicación.

RNF04. Integridad transaccional

Las ventas confirmadas y sus movimientos de inventario deberán mantener consistencia.

RNF05. Idempotencia

El reenvío de una operación previamente confirmada no deberá generar registros duplicados.

RNF06. Recuperación ante fallos

Las operaciones pendientes deberán conservarse ante errores de comunicación o respuestas fallidas del servidor.

RNF07. Mantenibilidad

El código deberá organizarse de forma que permita identificar las responsabilidades principales de presentación, negocio, persistencia y sincronización.

RNF08. Compatibilidad

La PWA deberá funcionar en navegadores modernos compatibles con sus tecnologías. La aplicación Flutter deberá conservar las capacidades móviles exigidas.

RNF09. Rendimiento

Las consultas de catálogo y reportes deberán evitar operaciones innecesariamente costosas. Los estudiantes deberán identificar y justificar sus decisiones de consulta y almacenamiento.

RNF10. Observabilidad

Los procesos de sincronización deberán registrar información suficiente para diagnosticar operaciones fallidas, sin exponer datos sensibles innecesariamente.

RNF11. Calidad

Se deberán implementar pruebas automatizadas que verifiquen reglas críticas del sistema.

RNF12. Coherencia documental

Los modelos UML deberán representar la solución realmente implementada.

11. TECNOLOGÍAS OBLIGATORIAS

Componente	Tecnología
Backend	Python + FastAPI
Frontend web	Angular
PWA	Service Worker, Web App Manifest, IndexedDB
Aplicación móvil	Flutter + Dart
Persistencia móvil	SQLite
Base de datos central	PostgreSQL
Integración	API REST
Inteligencia artificial	Modelo o servicio integrado mediante API
Modelado	UML 2.5+
Metodología	ICONIX
Despliegue	Nube, accesible externamente

Se permite utilizar bibliotecas auxiliares compatibles con estas tecnologías. No está permitido sustituir el desarrollo por plataformas o frameworks de comercio electrónico completos como Shopify, Magento, PrestaShop, WooCommerce o similares.

Las capacidades existentes de realidad aumentada y comercio electrónico deberán conservarse en el proyecto, aunque el segundo parcial no exige desarrollar un nuevo vestidor virtual.

DOCUMENTACION DE LA PLATAFORMA E-COMMERCE

TABLA DE CONTENIDO

Proyecto de Desarrollo de Software con ICONIX y UML

Índice resumido y estructurado para desarrollar un proyecto de software en **1 mes (4 semanas)**, priorizando el análisis, diseño, implementación y pruebas del sistema.

CAPÍTULO I. ASPECTOS GENERALES

1.1. Introducción

1.2. Planteamiento del problema

1.3. Objetivos del proyecto

1.3.1. Objetivo general

1.3.2. Objetivos específicos

1.4. Justificación

1.5. Alcance y limitaciones

1.6. Metodología de desarrollo ICONIX

CAPÍTULO II. ANÁLISIS DE REQUISITOS

2.1. Descripción del sistema propuesto

2.2. Identificación de actores

2.3. Requisitos funcionales y no funcionales

2.4. Reglas de negocio

2.5. Diagrama UML de casos de uso

2.6. Especificación de casos de uso

CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX

3.1. Modelo de dominio

3.2. Diagramas de robustez

3.3. Diagramas de secuencia UML

3.4. Diagrama de clases UML

3.5. Diseño de la base de datos

3.6. Diseño de interfaces de usuario

CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN

4.1. Herramientas y tecnologías utilizadas

4.2. Diseño de la arquitectura del sistema

4.3. Implementación de la base de datos

4.4. Desarrollo de los módulos del sistema

4.5. Integración de componentes

CAPÍTULO V. PRUEBAS Y RESULTADOS

5.1. Plan de pruebas

5.2. Pruebas funcionales

5.3. Pruebas de integración

5.4. Corrección de errores

5.5. Resultados de las pruebas

5.6. Casos de prueba

CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

6.2. Recomendaciones

7. BIBLIOGRAFIA

8. ANEXOS

SEGUNDO EXAMEN PARCIAL: SISTEMAS DE INFORMACION 2

MARTES DEFENSA 10 NOVIEMBRE 2026 Hrs.:7:00

PRESENTACION 1: DOMINGO 25/10/2026

PRESENTACION 2: DOMINGO 01/11/2026

PRESENTACION 3: DOMINGO 08/11/2026